DUOC UC — Escuela de Informática y Telecomunicaciones

EVALUACIÓN 2 — ARQUITECTURA, MODELO DE DATOS Y DIAGRAMAS

Plataforma CONEXSUR

Sistema Web de Gestión, Agendamiento y Pagos para Proveedor de Conectividad

Asignatura	Capstone
Docente	Cindy Betzabe Contador
Integrantes	Renato Huaiquil, Tomás Galdames
Versión	1.0 — octubre de 2026

N°	Artefacto	Ubicación
14	Documento de arquitectura	Sección 14
15	Modelo de base de datos	Sección 15 y archivo server/db/001_schema.sql
16	Diagramas: casos de uso, componentes, clases y despliegue	Sección 16
17	Software programado	Sección 17, repositorio y video de demostración

14. Documento de arquitectura

14.1 Propósito y alcance

Este documento describe cómo está construida la Plataforma CONEXSUR y por qué. Cubre el alcance del producto mínimo aprobado para la presentación de mediados de noviembre de 2026: autenticación por rol (HU-01), CRM de clientes (HU-03, HU-04), agendamiento técnico (HU-07, HU-08, HU-09), portal del cliente (HU-12, HU-13), pago con Mercado Pago (HU-14, HU-15) y despliegue (HU-21). Las historias restantes quedan como etapa posterior y no alteran la arquitectura.

14.2 Estilo arquitectónico

La solución es una aplicación web cliente-servidor en tres capas: cliente web (React), API (Express) y base de datos (PostgreSQL). La API es un monolito modular: un solo proceso, organizado en módulos por dominio (auth, clientes, visitas, boletas, pagos) con límites claros entre ellos.

Se descartaron los microservicios. Con dos desarrolladores y 4 a 5 semanas, el costo de operar varios servicios, redes internas y despliegues independientes no se compensa con ningún beneficio del producto. El monolito modular permite separar un módulo más adelante, porque cada uno accede a los datos solo mediante su repositorio.

14.3 Tecnologías y justificación

Capa	Tecnología	Justificación
Cliente	React 19, React Router, Tailwind CSS 4, Vite	Stack fijado en la propuesta. Interfaz responsiva y base para la PWA. Vite entrega compilación rápida y desarrollo con recarga.
API	Node.js 22, Express 5, zod	Stack fijado en la propuesta. zod valida y limpia toda entrada en un solo lugar por ruta.
Seguridad	helmet, cors, express-rate-limit, jsonwebtoken, scrypt (módulo crypto de Node)	Cabeceras seguras, origen restringido, límite de solicitudes, sesión firmada y hash de contraseñas sin dependencias nativas.
Datos	PostgreSQL 16, controlador pg	Restricciones declarativas (claves, unicidad, exclusión de rangos) y transacciones ACID para pagos y agenda.
Pagos	Mercado Pago Checkout Pro	Stack fijado en la propuesta. La pasarela captura los datos de tarjeta, por lo que el sistema nunca los toca.
Despliegue	Docker Compose, Nginx	Un solo comando levanta base de datos, API y cliente. Nginx entrega el cliente y reenvía /api a la API.
Pruebas	node:test (incluido en Node), Playwright para la prueba de punta a punta	Sin dependencias adicionales en el servidor; las pruebas usan una base de datos real.

14.4 Componentes y responsabilidades

La Figura 2 (sección 16.2) muestra los componentes. Cada petición recorre las capas siguientes:

Capa	Responsabilidad	No debe hacer
Middlewares	Autenticación (cookie con JWT), autorización por rol, validación con zod, verificación de Origin, límite de solicitudes y manejo central de errores.	Reglas de negocio.

Capa	Responsabilidad	No debe hacer
Rutas	Asocian cada URL con sus middlewares y su controlador.	Lógica de cualquier tipo.
Controlador	Traduce la petición HTTP a una llamada al servicio y la respuesta a JSON y código de estado.	Consultas SQL o reglas de negocio.
Servicio	Reglas de negocio, transacciones y auditoría. Ejemplo: cambiar el estado del servicio y registrar quién lo hizo en una sola transacción.	Conocer HTTP (req, res).
Repositorio	Todo el SQL, siempre parametrizado.	Reglas de negocio.
Adaptador Mercado Pago	Crear preferencias, consultar pagos y verificar la firma del aviso. Aísla a la pasarela del resto del sistema.	Modificar datos propios.

14.5 Estructura del repositorio

```
conexsur/
├── server/                     API Express
│   ├── db/                     Migraciones SQL versionadas (001_schema.sql, 002_semilla.sql)
│   ├── src/
│   │   ├── config/env.js       Variables de entorno validadas al iniciar
│   │   ├── db/                 pool.js (conexión y transacciones), migrar.js, semilla.js
│   │   ├── middlewares/        autenticar, autorizar, validar, origen, errores
│   │   ├── modulos/
│   │   │   ├── auth/           rutas, controlador, servicio, repositorio, esquemas
│   │   │   ├── clientes/       (mismo patrón)
│   │   │   ├── visitas/        agenda, disponibilidad, confirmación y control de cruces
│   │   │   └── planes/ tecnicos/ portal/ resumen/   (rutas de lectura)
│   │   ├── utilidades/         claves (scrypt), rut, auditoría, errores
│   │   ├── app.js              Construcción de la aplicación Express
│   │   └── index.js            Arranque y cierre ordenado
│   └── tests/                  Pruebas automáticas
├── client/                     React + Tailwind (Vite)
│   └── src/ (api, contexto, componentes, hooks, lib, paginas/{admin,tecnico,portal})
├── docs/                       Diagramas, capturas y video
└── docker-compose.yml          db + api + web
```

El módulo de pagos se agrega con el mismo patrón (rutas, controlador, servicio, repositorio y esquemas), lo que permite repartir el trabajo entre los dos integrantes sin que se pisen los archivos.

14.6 Decisiones de arquitectura

N°	Decisión	Motivo	Alternativa descartada
AD-1	Monolito modular	Equipo de 2 personas y plazo corto; menor costo de operación.	Microservicios: sobrecarga sin beneficio en este alcance.
AD-2	Sesión en cookie httpOnly con SameSite=Strict, más verificación de Origin	El JavaScript de la página no puede leer la sesión, lo que limita el robo por XSS. Strict y Origin mitigan CSRF.	Token en localStorage: legible por cualquier script inyectado.

N°	Decisión	Motivo	Alternativa descartada
AD-3	Contraseñas con scrypt y sal única por usuario	Función de derivación con costo de memoria, incluida en Node; evita compilar módulos nativos en el despliegue.	bcrypt o argon2: requieren dependencias nativas. El requisito RNF-01 admite cualquier hash robusto.
AD-4	El servidor define el monto y crea la preferencia de Mercado Pago	El navegador no puede alterar el valor a pagar.	Enviar el monto desde el cliente.
AD-5	El pago se confirma consultando la API de Mercado Pago, no por lo que informa el navegador ni por el cuerpo del aviso	La vuelta desde la pasarela y el cuerpo del webhook pueden falsificarse; la consulta directa con el token del servidor no.	Confiar en los parámetros de retorno.
AD-6	Idempotencia del pago: id de pago único en base de datos, un solo pago aprobado por boleta y registro del aviso por x-request-id	Mercado Pago puede reenviar avisos; no debe cobrarse ni marcarse dos veces.	Control solo en código de la aplicación.
AD-7	El cruce de horarios se impide con una restricción de exclusión en PostgreSQL	La base garantiza la regla aunque dos administradores confirmen a la vez.	Validar solo en el servicio: vulnerable a condiciones de carrera.
AD-8	Migraciones SQL versionadas y aplicadas por un script propio	Esquema reproducible en desarrollo, pruebas y producción.	Crear tablas a mano.
AD-9	Docker Compose con la base de datos sin puerto expuesto	Reduce la superficie de ataque; despliegue repetible.	Instalar cada servicio manualmente.

14.7 Flujo del pago con Mercado Pago (HU-14 y HU-15)

Este flujo es el más sensible del sistema. Se implementa en el Ciclo 2 (HU-14 y HU-15); aquí se fija el diseño. En el Ciclo 1 el botón Pagar de la boleta está deshabilitado con el aviso «disponible en el próximo ciclo».

1. El cliente pulsa Pagar en su boleta pendiente. El navegador envía solo el identificador de la boleta.
2. El servidor verifica que la boleta pertenece al cliente autenticado y está pendiente, toma el monto de la base de datos y crea un registro en pagos con estado iniciado y una referencia externa única.
3. El servidor crea la preferencia en Mercado Pago (monto, referencia externa, URL de retorno y URL del webhook) y guarda su identificador.
4. El servidor responde con la URL de pago y el navegador redirige al cliente a Checkout Pro. Los datos de tarjeta se ingresan solo en la pasarela.
5. Mercado Pago llama al webhook del servidor. El servidor valida la firma del aviso con el secreto configurado y guarda el evento (con su x-request-id único) antes de procesarlo.
6. El servidor consulta el pago en la API de Mercado Pago con su token. Si el estado es aprobado y el monto coincide con la boleta, abre una transacción: marca el pago como aprobado, la boleta como pagada y registra la auditoría.
7. Si el aviso llega repetido, la restricción de unicidad del id de pago impide procesarlo otra vez y el servidor responde 200 sin cambios.
8. Al volver a la aplicación, el cliente ve el estado leído desde la base de datos, no desde los parámetros de la URL. Si el aviso aún no llegó, se muestra pendiente de confirmación.

14.8 Seguridad

Control	Cómo se aplica	RNF	Estado
Contraseñas	scrypt con sal única; comparación en tiempo constante; mismo tiempo de respuesta si el correo no existe.	RNF-01	Implementado y probado
Fuerza bruta	Bloqueo de 15 minutos tras 5 fallos por cuenta y límite de solicitudes por IP.	RNF-03	Implementado y probado
Sesión	Cookie httpOnly, SameSite=Strict, Secure en producción; expira en 8 horas.	RNF-02	Implementado; HTTPS se activa en el despliegue
CSRF	SameSite=Strict, verificación de Origin y API que solo acepta JSON.	RNF-02	Implementado y probado
Entrada de datos	Validación zod en cuerpo y parámetros; consultas parametrizadas; límite de 100 KB por solicitud.	RNF-03	Implementado y probado
Autorización	Rol verificado en el servidor en cada ruta; el cliente web solo oculta pantallas.	RNF-06	Implementado y probado
Secretos	Variables de entorno validadas al iniciar; .env fuera del repositorio.	RNF-04	Implementado
Cabeceras y errores	helmet; mensajes de error sin detalles internos; trazas solo en el registro del servidor.	RNF-03	Implementado
Auditoría	Tabla auditoria con usuario, acción, entidad e IP para inicios de sesión y cambios de estado.	RNF-15	Implementado para auth, clientes y visitas; pagos en el Ciclo 2
Pagos	Monto del servidor, firma del webhook, verificación contra la API e idempotencia (sección 14.7).	RNF-05, RNF-14	Diseñado; se implementa en el Ciclo 2
Respaldos	Respaldo diario de PostgreSQL con retención de 7 días y restauración probada.	RNF-08	Pendiente (despliegue)
Calidad de código	ESLint y Prettier en el repositorio.	RNF-13	Pendiente

El estado de cada control se actualiza al cierre de cada ciclo. Los controles pendientes figuran en el backlog y no se declaran como cumplidos hasta que se verifiquen.

14.9 Despliegue y operación

- Tres contenedores: db (PostgreSQL 16 con volumen persistente), api (Node 22, usuario sin privilegios) y web (Nginx con el cliente compilado).
- Solo el contenedor web publica un puerto. La API y la base de datos se comunican por la red interna de Docker.
- La API aplica las migraciones pendientes al iniciar y valida su configuración; si falta una variable crítica, no arranca.
- Hosting: [COMPLETAR: proveedor o equipo donde se desplegará, con su costo y quién lo asume]. El proyecto no tiene financiamiento, por lo que se evalúa un plan gratuito o de bajo costo.
- HTTPS: certificado gratuito (Let's Encrypt) en el proxy. Mercado Pago exige HTTPS para el webhook; hasta tener dominio se prueba con el entorno de pruebas y un túnel temporal.

14.10 Trazabilidad con los requisitos no funcionales

Atributo	Requisitos	Táctica arquitectónica
Seguridad	RNF-01 a RNF-05	Capas de middleware, scrypt, cookie httpOnly, validación zod, SQL parametrizado, verificación de pagos.
Integridad	RNF-14	Transacciones, claves foráneas, restricción de exclusión para la agenda, índice único parcial para un solo pago aprobado por boleta.
Trazabilidad	RNF-15	Tabla auditoria y columnas de quién y cuándo en el cambio de estado del servicio.
Mantenibilidad	RNF-13	Módulos por dominio con el patrón rutas, controlador, servicio y repositorio; migraciones versionadas.
Escalabilidad	RNF-11	API sin estado (la sesión viaja en la cookie), índices en columnas de búsqueda.
Disponibilidad	RNF-08	Contenedores con reinicio automático, chequeo de salud de la base de datos, respaldo diario.

15. Modelo de base de datos

El modelo está implementado en PostgreSQL 16 y se aplica con la migración server/db/001_schema.sql. Se ejecutó sobre una base real y las restricciones se verificaron con las pruebas automáticas. Las convenciones son: nombres en español en snake_case, claves UUID, montos en pesos chilenos como enteros y fechas con zona horaria.

15.1 Tablas

Tabla	Propósito	Historias	Reglas destacadas
usuarios	Cuentas de administrador, técnico y cliente.	HU-01	Correo único sin distinguir mayúsculas; clave solo como hash; contador de intentos fallidos y bloqueo temporal.
planes	Planes de internet con velocidad y precio.	HU-12	Nombre único; precio y velocidad positivos.
clientes	Ficha del cliente y estado del servicio.	HU-03, HU-04	RUT único con formato validado; estado activo o cortado; guarda quién y cuándo cambió el estado.
visitas	Visitas técnicas solicitadas, confirmadas o realizadas.	HU-07, HU-08, HU-09	Código visible VT-00001 a partir de un número autoincremental; el fin es posterior al inicio; confirmar exige técnico; un técnico no puede tener dos visitas confirmadas que se crucen.
boletas	Cobro mensual por cliente.	HU-13	Una boleta por cliente y mes; monto positivo; estado pendiente, pagada o vencida.
pagos	Intentos de pago con Mercado Pago.	HU-14, HU-15	Referencia externa y id de pago de Mercado Pago únicos; solo un pago aprobado por boleta; el monto sale de la boleta.
eventos_mercadopago	Registro de cada aviso recibido de la pasarela.	HU-15	x-request-id único para no reprocesar; guarda el cuerpo y si la firma fue válida.
auditoria	Quién hizo qué y cuándo.	RNF-15	Usuario, acción, entidad, detalle en JSON e IP.

15.2 Reglas de integridad aplicadas en la base de datos

- **Exclusión de cruces de horario.** La restricción visitas_sin_cruce (extensión btree_gist) impide que un técnico tenga dos visitas confirmadas con rangos que se superpongan. La prueba de inserción de dos visitas cruzadas fue rechazada por la base.
- **Un solo pago aprobado por boleta.** Índice único parcial sobre pagos(boleta_id) donde estado es aprobado.
- **Idempotencia.** pagos.mp_pago_id y eventos_mercadopago.id_solicitud son únicos.
- **Formato del RUT.** Restricción CHECK con la expresión del formato sin puntos y con guion; el dígito verificador se valida en la aplicación (módulo 11).
- **Coherencia de estados.** Una visita confirmada o realizada exige técnico asignado.
- **Actualización de fechas.** Un disparador mantiene actualizado_en en las tablas que se modifican.

15.3 Trazabilidad con las historias del producto mínimo

Historia	Tablas que usa
HU-01 Iniciar sesión según rol	usuarios, auditoria
HU-03 Registrar cliente	clientes, planes, auditoria
HU-04 Consultar ficha y estado del servicio	clientes, planes, visitas, pagos, usuarios, auditoria
HU-07 Solicitar visita técnica	visitas, clientes
HU-08 Ver calendario técnico semanal	visitas, usuarios
HU-09 Asignar técnico y confirmar visita	visitas, usuarios, auditoria
HU-12 Ver estado del plan	clientes, planes
HU-13 Ver boleta pendiente y vencimiento	boletas
HU-14 Pagar con Mercado Pago	boletas, pagos
HU-15 Confirmar pagos por notificación verificada	pagos, boletas, eventos_mercadopago, auditoria

El script completo está en server/db/001_schema.sql y debe adjuntarse a la entrega junto con este documento. El diagrama entidad-relación (Figura 1) está en la página siguiente.

15.4 Diagrama entidad-relación

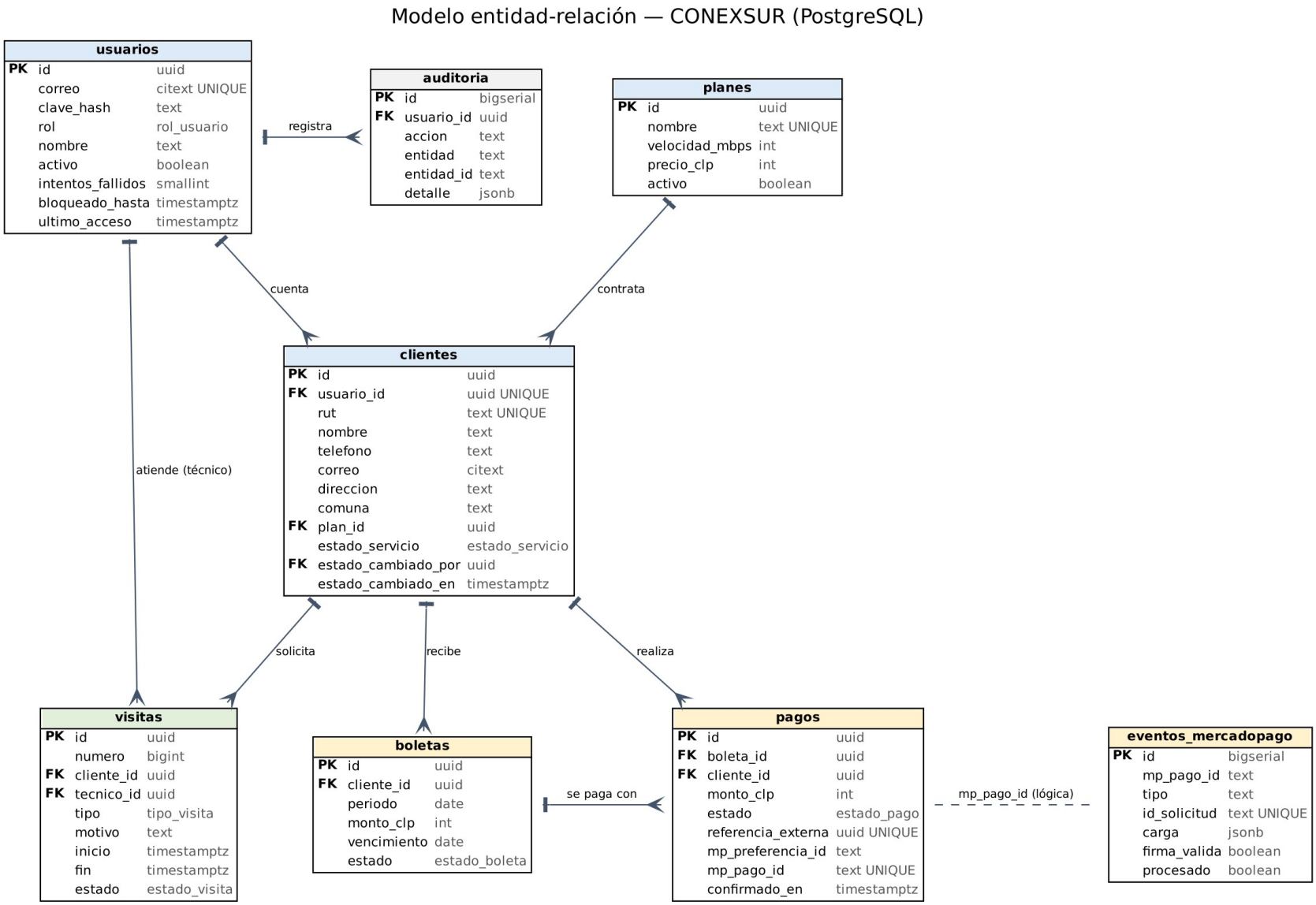


Figura 1. Modelo entidad-relación

16. Diagramas

16.1 Diagrama de casos de uso

Actores: administrador, técnico, cliente y Mercado Pago (sistema externo). Cada caso de uso de negocio lleva el código de su historia. Los casos con «include» y «extend» muestran pasos internos relevantes: crear la preferencia de pago, consultar el pago a la API y validar el cruce de horario.

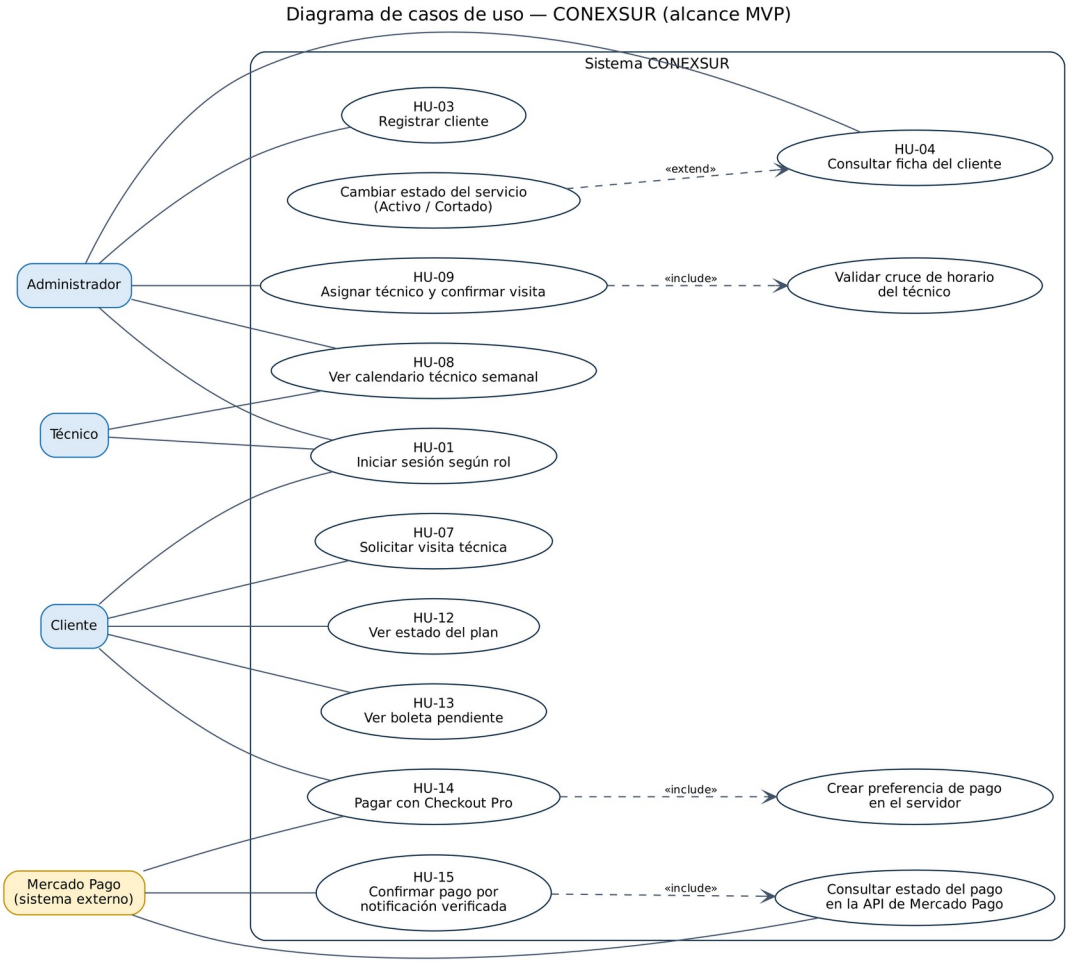


Figura 2. Casos de uso del producto mínimo

16.2 Diagrama de componentes

Muestra el cliente web, la API con sus middlewares y módulos, la base de datos y la pasarela de pagos. La API recibe el aviso de Mercado Pago y consulta su API; el cliente web solo habla con la API.

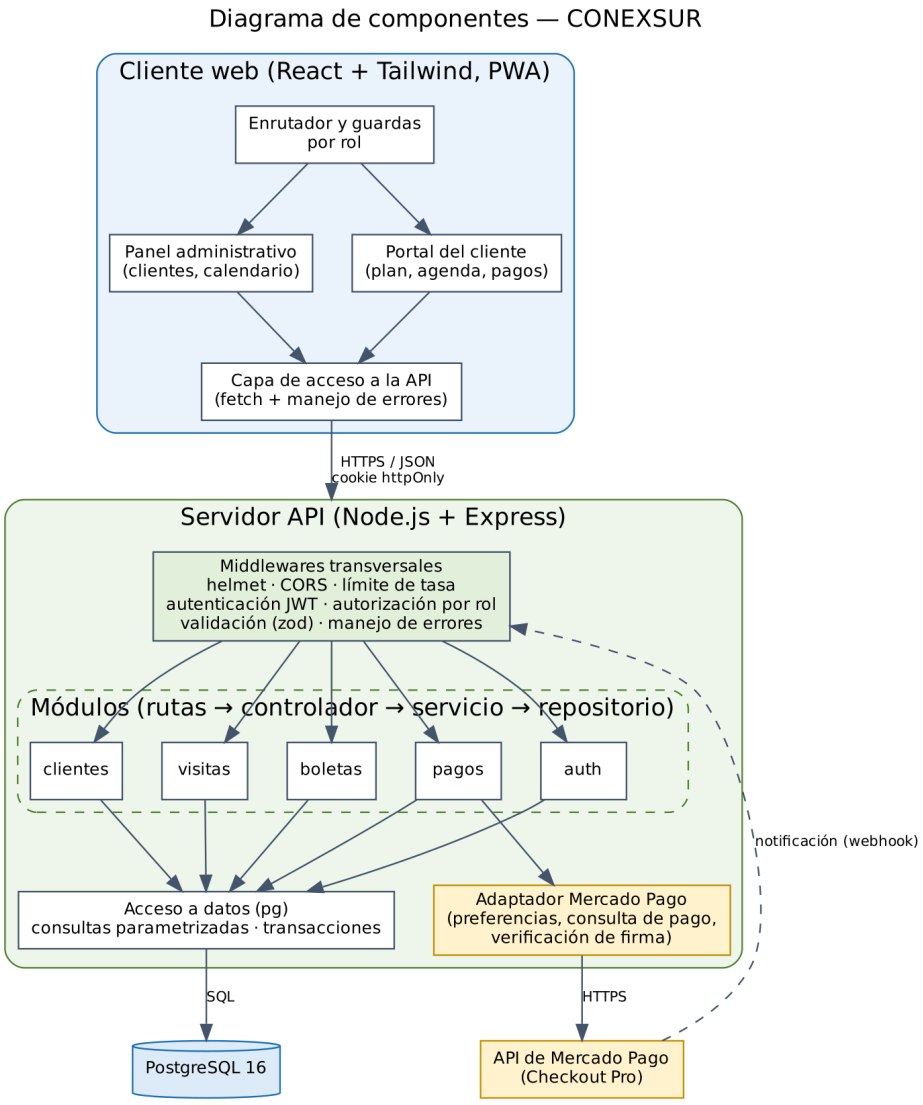


Figura 3. Componentes del sistema

16.3 Diagrama de clases

Representa el modelo de dominio y sus relaciones. Es un diagrama conceptual: en el código, el comportamiento de estas clases vive en los servicios de cada módulo como funciones, no como clases con estado. Las entidades coinciden con las tablas de la sección 15.

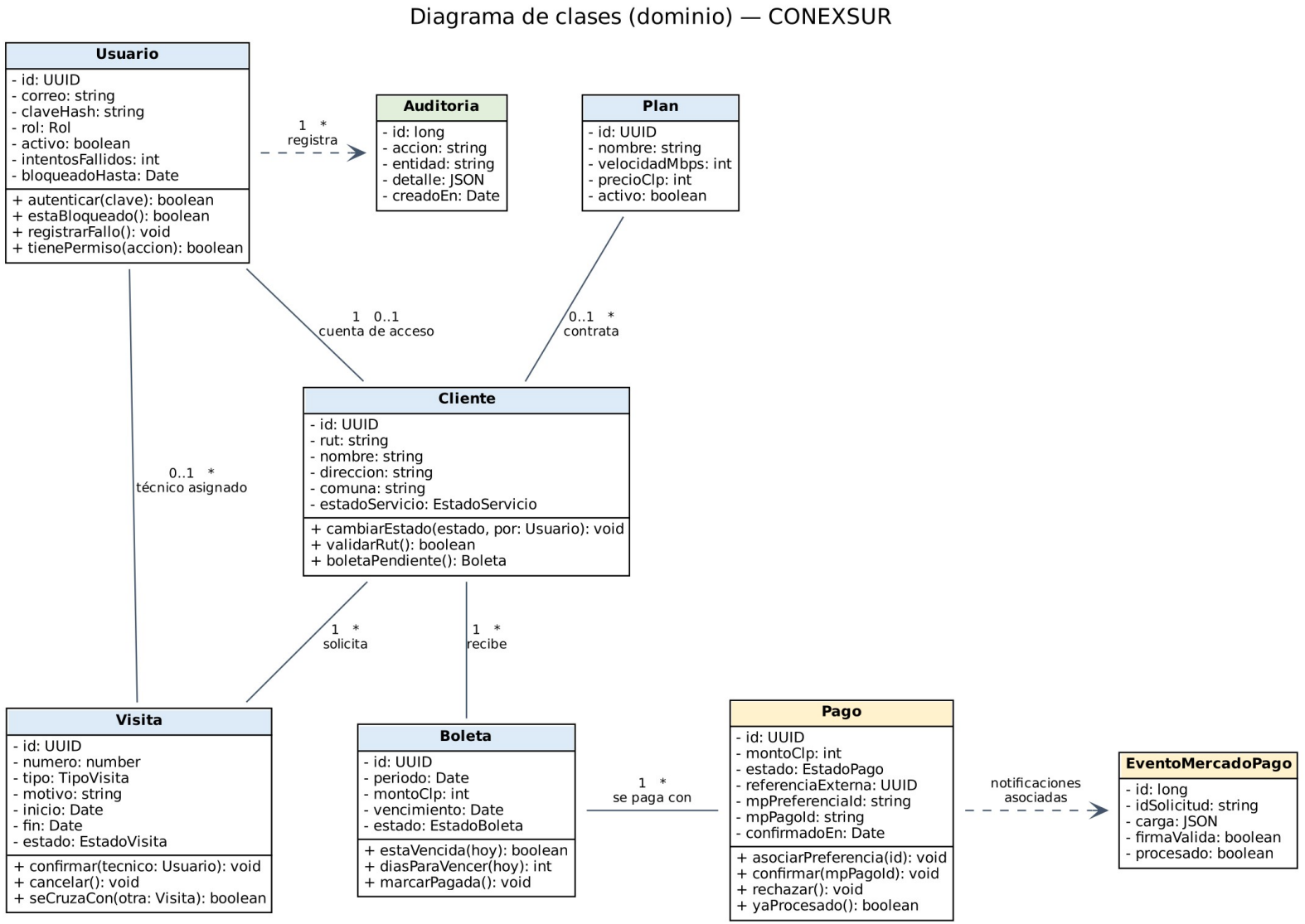


Figura 4. Modelo de dominio

16.4 Diagrama de despliegue

Un servidor con Docker Compose aloja tres contenedores. El navegador accede por HTTPS al proxy; Mercado Pago llama al webhook de la misma entrada. La base de datos no es accesible desde fuera.

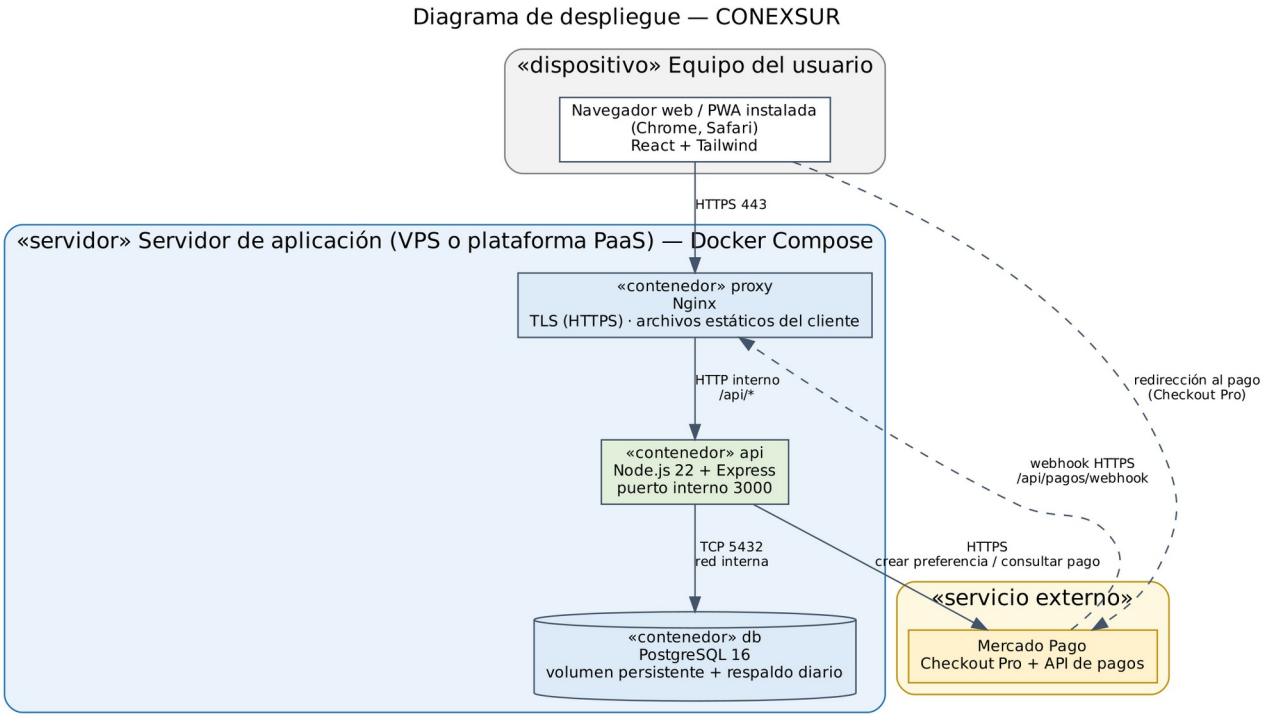


Figura 5. Despliegue

17. Software programado: evidencia

El código está en el repositorio del proyecto (carpetas server y client) y se entrega con un video de demostración de unos 80 segundos (docs/demo_conexsur.mp4). Lo construido corresponde al Ciclo 1 del documento de gestión: las historias HU-01, 03, 04, 07, 08 y 09, y HU-12 y 13 en modo lectura. Las tarjetas correspondientes están en Pruebas (QA), a la espera de la revisión cruzada entre los integrantes.

17.1 Qué funciona

Historia	Alcance entregado	Verificación
HU-01	Inicio y cierre de sesión por rol (administrador, técnico y cliente), protección de rutas en servidor y cliente, bloqueo por intentos fallidos.	7 pruebas automáticas y prueba en navegador.
HU-03	Registro de clientes con validación del RUT (módulo 11), detección de duplicados y selección de plan.	Pruebas automáticas y prueba en navegador.
HU-04	Listado, ficha del cliente con plan, visitas y pagos, y cambio de estado del servicio con registro de quién y cuándo.	Pruebas automáticas y prueba en navegador.
HU-07	El cliente agenda una visita en 2 pasos desde el portal móvil: ve solo las franjas libres (lunes a sábado, de 09:00 a 17:00, con colación) y recibe un código VT-00001. La reserva usa un bloqueo para que dos clientes no tomen la última franja a la vez.	11 pruebas automáticas de visitas y prueba en navegador.
HU-08	Calendario semanal: grilla por día y hora en escritorio y agenda por día en teléfono, con filtro por técnico. El técnico solo ve sus visitas.	Prueba automática de permisos y prueba en navegador.
HU-09	El administrador ve las solicitudes por confirmar, asigna un técnico y confirma. La base rechaza dos visitas confirmadas que se crucen para el mismo técnico (error 409 en la API).	Prueba automática del cruce y prueba en navegador.
HU-12 y HU-13	Inicio del portal con plan, velocidad, estado del servicio y alerta cuando la boleta vence en 5 días o menos; pantalla de pagos con boleta pendiente e historial. El botón Pagar está deshabilitado hasta el Ciclo 2.	Prueba automática del portal y prueba en navegador.
Base técnica	Esquema de base de datos, migraciones, datos de demostración, Docker Compose, validación de configuración y patrón de módulos.	Esquema aplicado en una base real.

Resultado de las pruebas automáticas del servidor: 27 pruebas, 27 aprobadas (npm test, sobre una base de datos real de pruebas). La prueba en navegador recorre el login, el alta de un cliente con RUT válido e inválido, la solicitud de una visita desde el teléfono, su confirmación por el administrador y la verificación de que un cliente no pueda abrir el panel de administración.

17.2 Qué falta

Quedan por construir el pago con Mercado Pago (HU-14 y HU-15, Ciclo 2) y el despliegue con la prueba de aceptación del cliente (HU-21, Ciclo 3), según el plan de la sección 12 del documento de gestión. También quedan ESLint y Prettier, los respaldos de la base de datos y la prueba de restauración. Docker Compose está escrito y su sintaxis validada, pero aún no se levantó en un servidor.

17.3 Capturas

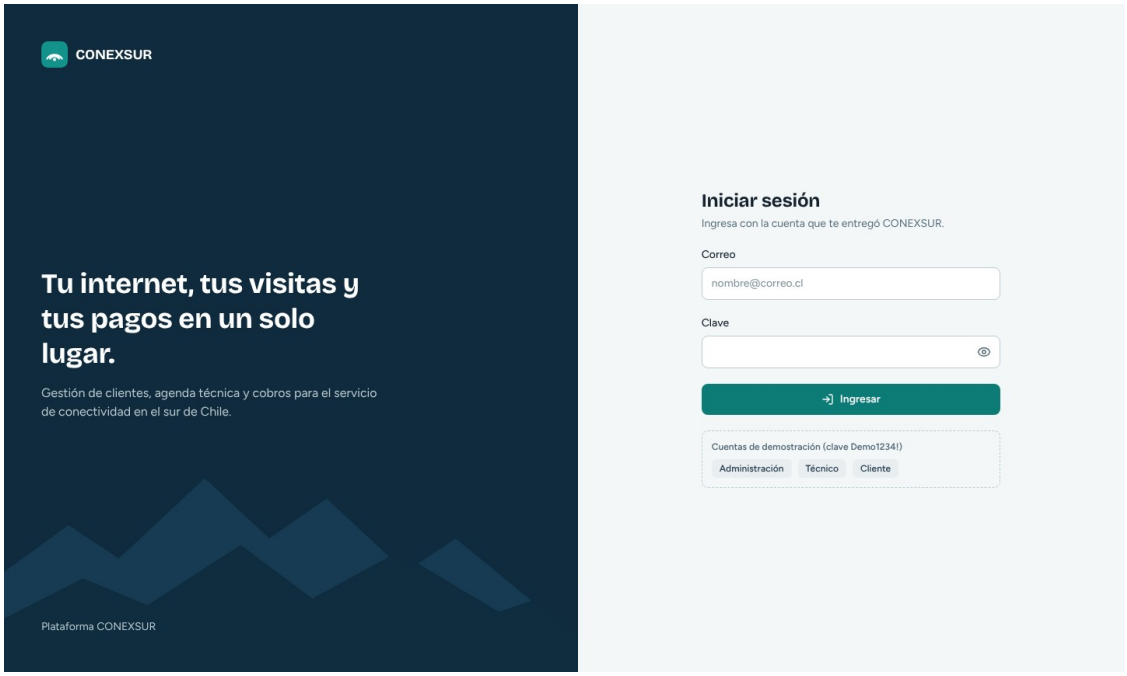


Figura 6. Inicio de sesión

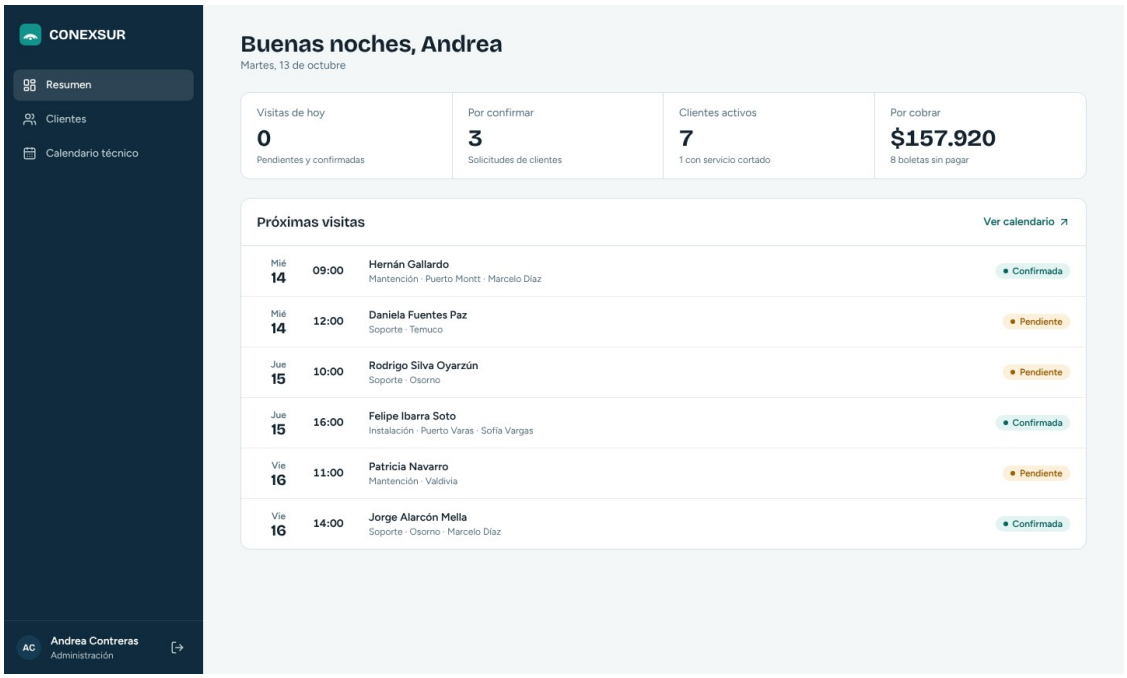


Figura 7. Resumen del administrador

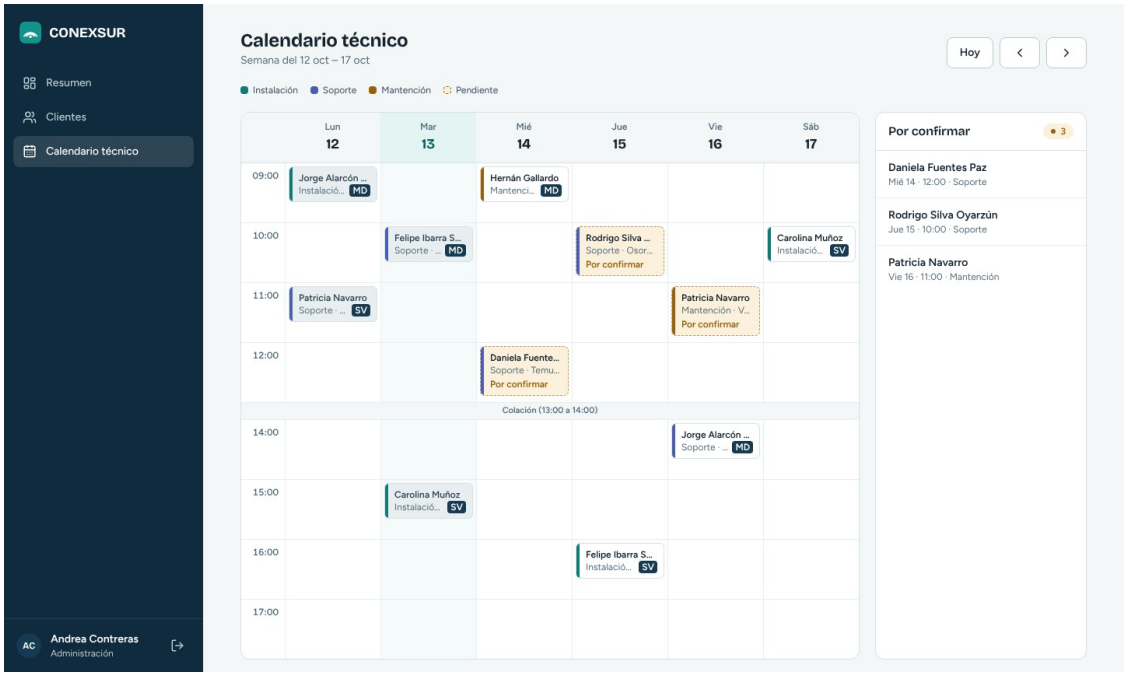


Figura 8. Calendario técnico semanal con solicitudes por confirmar

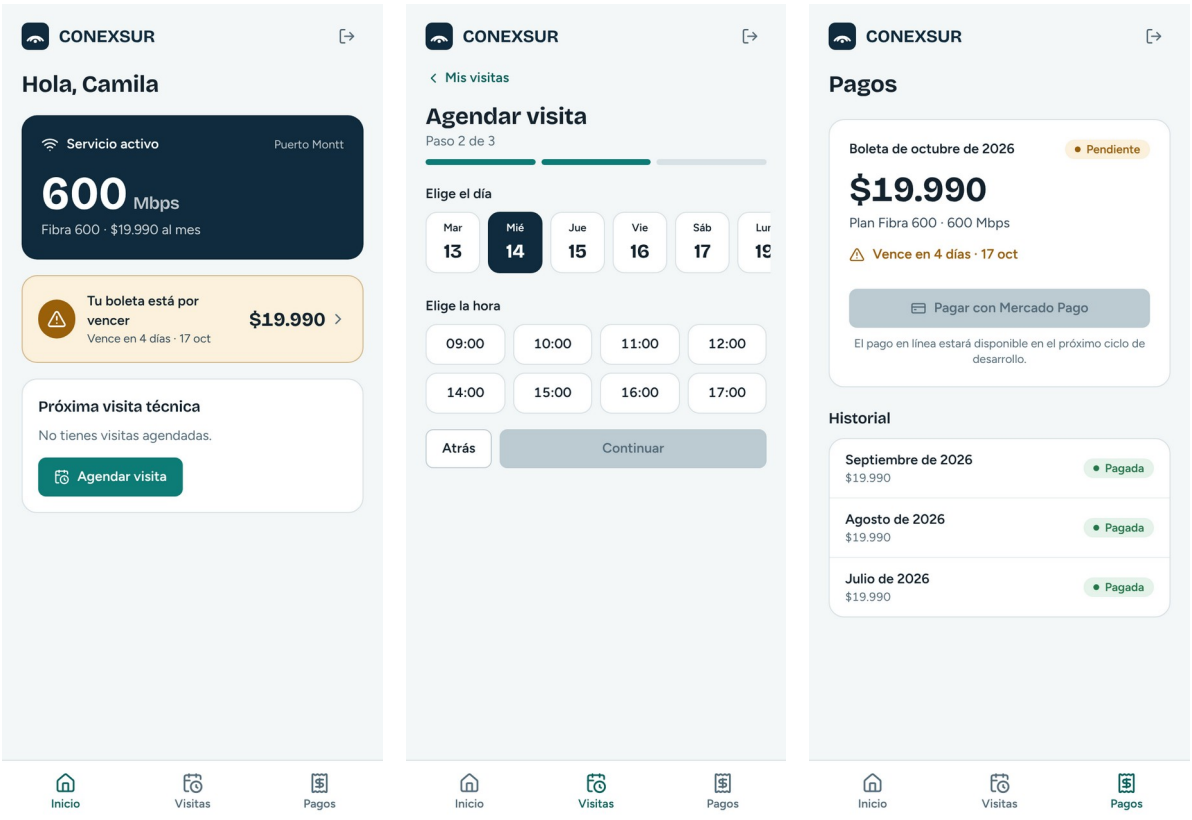


Figura 9. Portal del cliente en el teléfono: inicio, agendar una visita (paso 2) y pagos

17.4 Cómo ejecutar el sistema

```
createdb conexsur && cd server
cp .env.example .env        # completar DATABASE_URL y JWT_SECRETO
npm install && npm run migrar && npm run semilla && npm run dev
npm test                    # 27 pruebas (usa la base conexsur_test)

cd ../client && npm install && npm run dev      # http://localhost:5173

Usuarios de demostración (clave Demo1234!):
admin@conexsur.cl · tecnico@conexsur.cl · cliente@conexsur.cl
```

El paso a paso para instalar las herramientas, subir el código a GitHub y armar el tablero está en docs/GUIA_INICIO.md.